Assignment 4: Fact Checking Claims with AVeriTeC

Academic Honesty Please see the course syllabus for information about collaboration in this course. While you may discuss the assignment with other students, **all work you submit must be your own!**

Goals The primary goal of this assignment is to give you hands on experience with automatic fact verification. You will decide whether real world claims are supported or refuted by supplied evidence, first with word overlap features and then with a pretrained textual entailment model. In Part 3 you will code a method that identifies the verification questions needed to check a claim. The three parts let you study evidence interpretation and question decomposition without building a document retrieval system.

1 Dataset and Code

IMPORTANT: Use Python 3.10 and the package versions in `requirements.txt`. The grading environment runs on a CPU. Create an environment and install the dependencies with:

```
python3.10 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The first local entailment run may download the pinned model checkpoint. The grading environment has that checkpoint preinstalled, so your submission must not download resources during grading. Native Intel macOS does not have a wheel for the pinned `torch==2.5.0` version. Intel Mac users should run the pinned environment under Linux. Apple Silicon users should use a native ARM Python installation rather than Python running through Rosetta.

Data This assignment uses a selected subset of AVeriTeC [Schlichtkrull et al., 2023], a dataset of real world claims accompanied by verification questions and answers. AVeriTeC has four verdict categories; this course task keeps only **Supported** and **Refuted**. The development file `data/dev.jsonl` contains 260 labeled claims, balanced at 130 per label. A separate 200 claim balanced set is held out by course staff for evaluation. These are distinct claims and retrieval has already been done. Each claim comes with one or more question answer evidence items. You do not need to search Wikipedia or the web.

Each line of the development file is a JSON object with a claim, its label, and a list of passages. A passage contains a verification question, its answer, and the answer type. The driver derives `title` from the question and `text` from the answer for the existing `predict(fact, passages)` interface; these duplicate fields are not stored in the data. The following example is shortened from the development data:

```
{"claim": "The Kenyan judiciary does not even get 1% of the national budget.",
 "label": "Supported",
 "passages": [
   {"question": "When was the claim made?", "answer": "2021"},
   {"question": "How much did the judiciary receive in 2021?",
    "answer": "0.6% of the national budget."}
 ]}
```

Boolean answers may also have an explanation, which is part of the evidence. Development labels are used for local scoring and are not passed to your checker. An evidence item (question) can be unanswerable, but that alone does not imply a Refuted label. Some claims require comparing numbers or combining several answers. A single answer may be too short to interpret without its question.

Framework code The supplied files are organized as follows:

1. `factchecking_main.py` loads examples, constructs a checker, runs it, and reports accuracy and macro-F1. You should not modify this driver for submission.

2. `factcheck.py` contains the `FactExample` and `EntailmentModel` classes and the three methods you will implement. This is the only file you should submit.

3. `question_decomposition_main.py` is the Part 3 development driver. `data/questions_dev.jsonl` supplies 64 labeled question selection tasks. `README.md` summarizes the interfaces, data fields, and local commands.

Getting started Run the two simple baselines:

```
python factchecking_main.py --mode random
python factchecking_main.py --mode always_refuted
```

A constant Refuted prediction obtains 50% accuracy on this balanced development set but only one-third macro-F1. Macro-F1 computes F1 separately for Supported and Refuted, then averages the two values. This gives each label equal weight and makes it harder to earn credit by predicting only one class.

2 Part 1: Word Overlap (40 points)

A supported claim might share words with the evidence, whereas a refuted claim might be missing a key term. Build a non-neural lexical method in `WordOverlapFactChecker.predict(fact, passages)`. You can choose tokenization, stemming, stopword handling, negation and number features, overlap scores, and how to combine several passages. You may tune weights or fit a small non-neural feature combiner on the labeled development set, but inference must be self contained in `factcheck.py`. The grader will not run a separate training script or load an extra weights file.

For Part 1 the driver deliberately passes **answer-side evidence only**: the gold verification question wording is removed before `predict` is called. Otherwise, a question that repeats the claim could create misleadingly high word overlap. Boolean answers and explanations are still available. Your method should use the evidence it receives and return exactly `Supported` or `Refuted`. Run:

```
python factchecking_main.py --mode word_overlap
```

A high overlap does not guarantee support. Changing one number or negating one relation can reverse the verdict while leaving most words unchanged. Aim for at least **0.63 macro-F1** on the held out claims for full credit. Assignment points scale linearly from zero at 0.50 macro-F1 to 40 at 0.63, with scores outside that interval capped at zero or 40. A balanced random predictor averages about 0.50. Be sure to keep the 200 claim grading run under one minute on a CPU, including initialization. The grader verifies the output labels and performance.

3 Part 2: Textual Entailment (40 points)

Textual entailment models compare a premise with a hypothesis and estimate whether the premise entails, contradicts, or is neutral toward the hypothesis. For this part, use the supplied DeBERTa-v3 base NLI checkpoint [Laurer]. The model wrapper in `factcheck.py` accepts premises and the claim, then returns probabilities for the three NLI classes. Implement `EntailmentFactChecker.predict(fact, passages)` to turn those model outputs into a Supported or Refuted decision.

Using the entailment model A bare answer such as "Yes," "No," or "0.6%" is often uninterpretable as a premise. Combine verification questions with their answers before calling the NLI model. Some answers only make sense together because separate items may establish the date, subject, and quantity needed to interpret a claim. You may score a joined question answer premise, individual question answer premises, or both. Compare these choices on the development set and consider whether truncation removes relevant evidence. The method must work with a varying number of passages, including a single passage. Do not assume a fixed evidence count.

Runtime NLI inference is much more expensive than string overlap. Score at most seven NLI premise claim pairs per claim: up to six answerable evidence items and one optional joined bundle. **The limit is seven premise claim pairs per claim, summed across all NLI calls. Requesting an eighth pair on any claim causes the entire Part 2 run to receive zero points. Using fewer than seven is allowed.** Use the supplied wrapper for all NLI inference; do not change its implementation, reset its budget inside `predict`, or call the underlying model directly. Keep the model on CPU and avoid repeatedly loading it. The wrapper truncates premise claim pairs to 256 tokens. The run budget is five minutes and approximately 6 GB peak memory. Your code must not require a network connection, retrieval, or model training during grading. Run:

```
python factchecking_main.py --mode entailment
```

Aim for at least **0.82 macro-F1** on the held out claims for full credit. Assignment points scale linearly from zero at 0.50 macro-F1 to 40 at 0.82, with scores outside that interval capped at zero or 40. The check also includes output format and the CPU runtime limit.

4 Part 3: Verification Question Decomposition (20 points)

A claim can require more than one fact to verify. For example, a claim comparing two years needs a value for each year before one can make the comparison. In this part you will code a constrained question-decomposition method. Each task gives one claim and six candidate verification questions. Implement `QuestionDecompositionFactChecker.select_questions(claim, candidates)` to return the IDs of exactly **two distinct questions** that best cover its central verification facts. The two questions form a plan before any answers are known.

The method receives only the claim and each candidate's ID and question text. It does not receive candidate answers, verdict labels, sources, or the answer key. Prefer specific, independently answerable questions that together cover the relevant entities, dates, quantities, and comparisons. For this exercise, a question that merely asks "Is the entire claim true?" does not count as decomposition; a question about a related but different date, place, or measure is insufficient. You are selecting questions from a supplied list rather than writing new questions or retrieving evidence.

For example, imagine the claim "A university enrolled more students in 2022 than in 2021." The questions "How many students enrolled in 2021?" and "How many students enrolled in 2022?" supply the two quantities needed for the comparison. "Was enrollment higher in 2022 than in 2021?" simply restates the whole claim; "How many students enrolled in 2020?" asks for an unnecessary quantity. Your function should distinguish such cases by comparing the claim with the meaning of each candidate question, not by looking up a memorized task.

A good implementation may extract claim entities, time expressions, quantities, and relations, then score how well pairs of candidate questions cover those elements. It should also penalize irrelevant changes of

entity or date and questions that directly ask for the final verdict. Course staff authored these candidate choices and keys from AVeriTeC claims; the original dataset annotations are preserved for Parts 1 and 2. You may design another approach, provided that the method uses the permitted text and returns two valid IDs. Candidate order is deliberately varied in the supplied tasks, so positional rules are not a dependable solution. Different claims may share the same candidate questions while requiring different selections; use the claim to identify the facts that matter.

Development and scoring The file `data/questions_dev.jsonl` contains 64 labeled tasks with gold question IDs for local development. A separate 64 task set is held by staff. Run:

```
python question_decomposition_main.py \
    --data_path data/questions_dev.jsonl
```

The local driver reports question selection F1. Each task has two gold questions and you must select two; therefore precision, recall, and F1 all equal the fraction of gold questions selected. Selecting one gold question and one distractor earns 0.5 for that task. A random two-of-six selection averages one-third F1. Full credit begins at **0.70 F1** on the held out tasks, which means selecting at least 90 of the 128 required questions. Scores between one-third and 0.70 scale linearly from zero to 20 points. Invalid selections earn zero on the affected task. Keep the 64 task run under one minute on a CPU.

5 Deliverables and Submission

Code Submission Submit one `factcheck.py` file for all three parts. Preserve the provided method signatures, exact labels, and NLI wrapper. You may add helper functions, but do not hard code development or evaluation claims, read gold answer fields inside a checker, or infer correct questions from candidate positions. Make sure the three commands below run locally before submission:

```
python factchecking_main.py --mode word_overlap
python factchecking_main.py --mode entailment
python question_decomposition_main.py \
    --data_path data/questions_dev.jsonl
```

Each part runs independently. An uncaught exception, invalid verdict label, or timeout causes that part to earn zero without changing the other two parts' scores. Part 3 handles an invalid question selection as zero for that task. The development data are for experimentation. Final scores use separate held out claims and question tasks. **The displayed performance targets and limits are the course staff draft for this refactor and will be confirmed before assignment release.**

References

M. Laurer. Deberta-v3-base-mnli-fever-anli model checkpoint. URL `https://huggingface.co/MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli`.

M. Schlichtkrull, Z. Guo, and A. Vlachos. Averitec: A dataset for real-world claim verification with evidence from the web. *Advances in Neural Information Processing Systems*, 36:65128–65167, 2023. URL `https://fever.ai/dataset/averitec.html`.